51

BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Tek Bir Döngü, Özel Komut Yok: Neden /bug Komutu Bulunmaz?

One Loop, No Special Commands: Why There is No /bug Command in AIUP

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                NEDEN /BUG VEYA /CHANGE KOMUTU YOKTUR?


Geleneksel Yanılgı:
- "Yeni özellik için /create-feature çalıştır!"
- "Bug için /bug-fix çalıştır!"
- "Değişiklik için /change-request çalıştır!"

❌ BU NEDEN YANLIŞTIR?:
Çünkü sistemin kod üretim motoru (/implement) ne tür bir iş yaptığınızı
umursamaz! Motor sadece GÜNCEL ŞARTNAMEYİ ve TESTLERİ okur!

✅ AIUP TEK DÖNGÜSÜ (ONE LOOP TO RULE THEM ALL):

  1. Niyet Belirleme: İmza testiyle durumu analiz edin.
  2. Şartnameyi Değiştir: UC-XXX dosyasında kuralı veya akışı güncelleyin!
  3. Aynı Komutu Koş: `/implement UC-XXX`
  4. Testler Güvenceye Alır: Değişen testler güncellenir, eski testler
     yeşil kalmaya devam eder!

🎯 NETİCE: Özel bir sihirli komut yoktur; şartname değişir, kod hizalanır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir 3D yazıcıyı düşünün: 3D yazıcıya 'Bu bir heykel', 'Bu kırık bir dişli tamiri' veya 'Bu yeni bir vazo' diye ayrı komutlar vermezsiniz. 3D yazıcıya sadece güncel 3D çizim dosyasını (CAD / Şartname) verirsiniz ve 'Yazdır' (Print / Implement) dersiniz! Yazıcı dosyadaki farkı kendisi basar. AIUP'ta da tek komut 'Hizala ve Üret'tir!

🇬🇧 ENGLISH (ELI5)

Consider an industrial 3D printer: You don't possess separate buttons for 'Print art', 'Print replacement gear', or 'Print vase'. You simply feed it the updated 3D CAD blueprint (The Specification) and press 'Print' (/implement)! The printer materializes the exact delta. In AIUP, there is only one universal generation loop.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Tek Bir Döngü İlkesi (One Loop, No Special Commands): Birçok AI destekli araç 'Fix Bug', 'Refactor', 'Add Feature' gibi onlarca farklı özel prompt veya komut icat etmeye çalışır. Simon Martinelli bu yaklaşımı reddeder: AI Unified Process'te ne bir `/bug` komutu ne de bir `/change-request` komutu vardır! İşin türü hangi komutu çalıştıracağınıza değil, şartnamenin neresini düzenleyeceğinize karar verir. 1) Sınıflandırma yapılır. 2) Gereksinim mühendisi şartnameyi günceller (örneğin kural metnini veya alternatif akış adımını düzeltir). 3) Her zamanki `/implement UC-XXX` komutu yeniden çalıştırılır. Üretim motoru mevcut Java sınıflarını (Controller, Entity, Repository, Test) okur ve şartnamedeki değişikliği yerinde uzlaştırır (reconciliation). 4) Testler güncellenen beklentiyi karşılar; kaldırılan senaryoların testleri silinir. Bu sayede hata çözümü geçici bir yama üretmek yerine, sistemin en değerli varlığı olan şartnameyi zenginleştiren kalıcı bir yatırıma dönüşür.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Single Uniform Loop Principle: Fragmented tooling often introduces divergent commands for defects versus enhancements (`/fix-bug`, `/patch`, `/refactor`). Martinelli explicitly rejects this fragmentation: there is no `/bug` or `/change-request` command in the AI Unified Process. The nature of the work dictates what you edit in the specification, never which CLI command you invoke. 1) Classify by intent via the sign-off test. 2) Update the specification first (correcting a business rule or refining a flow trigger). 3) Rerun the identical `/implement UC-XXX` command. The generator parses existing source files, inspects Git diffs, and reconciles implementations in place without duplicate scaffolding. 4) Tests carry the delta. Existing invariants remain green while modified assertions guard the new behavior. Fixing a defect becomes an asset enhancement rather than an unrecorded runtime patch.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Yerinde Uzlaştırma (In-Place Reconciliation) Mantığı
package com.maang.aiup.reconcile;

public class SpecReconciliationDemo {
    // Şartname: "Sistem parolanın en az 12 karakter olduğunu doğrular (BR-009)"
    // Eski Kod: 8 karakterdi (Hata tespit edildi ve şartnamede 12 olarak düzeltildi)
    public static class PasswordSecurityPolicy {
        public static boolean isPasswordStrong(String password) {
            // /implement komutu şartnamedeki BR-009'u okur ve sabiti 12 yapar:
            int minimumLength = 12; // Reconciled from spec!
            return password != null && password.length() >= minimumLength;
        }
    }

    void main() {
        System.out.println("8 Karakterli Şifre Geçerli mi?: " +
PasswordSecurityPolicy.isPasswordStrong("Pass1234"));
        System.out.println("14 Karakterli Şifre Geçerli mi?: " +
PasswordSecurityPolicy.isPasswordStrong("SuperStrongPass2026!"));
        System.out.println("✅ /implement şartnameyi okudu ve kodu yerinde güncelledi.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Hata düzeltirken test dosyasındaki assertion'ı doğrudan değiştirmeyin; assertion şartnameden türer. Önce use case şartnamesini değiştirin, testi şartname güncellesin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never alter an automated assertion directly during a bug fix; the assertion derives from the spec. Rectify the specification first, then reconcile the test suite.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- AIUP'ta /bug veya /patch komutu yoktur; tek bir evrensel üretim döngüsü vardır.
- İşin türü çalıştırılan komutu değil, şartnamede düzenlenen alanı belirler.
- Hata çözümleri şartnameye işlendiği için aynı hatanın tekrar oluşması imkansızdır.

🎯 Key Architectural Takeaways

- There is no /bug command in AIUP; all changes traverse the single specification loop.
- Work classification dictates which spec section to edit, not which command to run.
- Defects fixed through specs permanently inoculate the codebase against regressions.